

第 10 章

Canny 边缘检测

Canny 边缘检测是一种使用多级边缘检测算法检测边缘的方法。1986 年，John F. Canny 发表了著名的论文 *A Computational Approach to Edge Detection*，在该论文中详述了如何进行边缘检测。

OpenCV 提供了函数 `cv2.Canny()` 实现 Canny 边缘检测。

10.1 Canny 边缘检测基础

本节对 Canny 边缘检测的步骤进行简单的介绍，希望能够帮助大家理解 Canny 边缘检测的原理。

Canny 边缘检测分为如下几个步骤。

步骤 1：去噪。噪声会影响边缘检测的准确性，因此首先要将噪声过滤掉。

步骤 2：计算梯度的幅度与方向。

步骤 3：非极大值抑制，即适当地让边缘“变瘦”。

步骤 4：确定边缘。使用双阈值算法确定最终的边缘信息。

下面对上述步骤分别进行简单的介绍。

1. 应用高斯滤波去除图像噪声

由于图像边缘非常容易受到噪声的干扰，因此为了避免检测到错误的边缘信息，通常需要对图像进行滤波以去除噪声。滤波的目的是平滑一些纹理较弱的非边缘区域，以便得到更准确的边缘。在实际处理过程中，通常采用高斯滤波去除图像中的噪声。

图 10-1 演示了使用高斯滤波器 T 对原始图像 O 中像素值为 226 的像素点进行滤波，得到该点在滤波结果图像 D 内的值的过程。

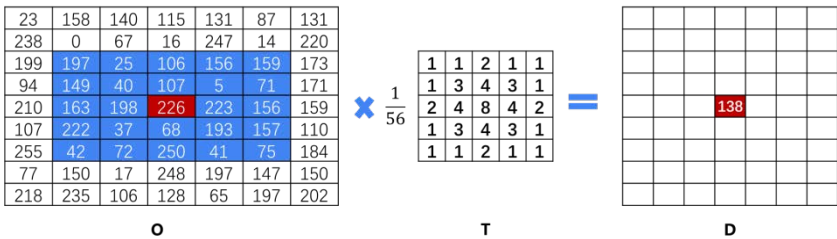


图 10-1 高斯滤波示例



在滤波过程中，我们通过滤波器对像素点周围的像素计算加权平均值，获取最终滤波结果。对于高斯滤波器 T，越临近中心的点，权值越大。在图 10-1 中，对图像 O 中像素值为 226 的像素点，使用滤波器 T 进行滤波的计算过程及结果为：

$$\begin{aligned} \text{结果} &= \frac{1}{56} \times (197 \times 1 + 25 \times 1 + 106 \times 2 + 156 \times 1 + 159 \times 1 \\ &\quad + 149 \times 1 + 40 \times 3 + 107 \times 4 + 5 \times 3 + 71 \times 1 \\ &\quad + 163 \times 2 + 198 \times 4 + 226 \times 8 + 223 \times 4 + 156 \times 2 \\ &\quad + 222 \times 1 + 37 \times 3 + 68 \times 4 + 193 \times 3 + 157 \times 1 \\ &\quad + 42 \times 1 + 72 \times 1 + 250 \times 2 + 41 \times 1 + 75 \times 1) \\ &= 138 \end{aligned}$$

当然，高斯滤波器（高斯核）并不是固定的，例如它还可以是：

$$B = \frac{1}{159} \begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}$$

滤波器的大小也是可变的，高斯核的大小对于边缘检测的效果具有很重要的作用。滤波器的核越大，边缘信息对于噪声的敏感度就越低。不过，核越大，边缘检测的定位错误也会随之增加。通常来说，一个 5×5 的核能够满足大多数的情况。

2. 计算梯度

在第 9 章中，我们介绍了如何计算图像梯度的幅度。在这里，我们关注梯度的方向，梯度的方向与边缘的方向是垂直的。

边缘检测算子返回水平方向的 G_x 和垂直方向的 G_y 。梯度的幅度 G 和方向 θ （用角度值表示）为：

$$\begin{aligned} G &= \sqrt{G_x^2 + G_y^2} \\ \theta &= \text{atan2}(G_y, G_x) \end{aligned}$$

式中， $\text{atan2}(\cdot)$ 表示具有两个参数的 $\arctan$ 函数。

梯度的方向总是与边缘垂直的，通常就近取值为水平（左、右）、垂直（上、下）、对角线（右上、左上、左下、右下）等 8 个不同的方向。

因此，在计算梯度时，我们会得到梯度的幅度和角度（代表梯度的方向）两个值。

图 10-2 展示了梯度的表示法。其中，每一个梯度包含幅度和角度两个不同的值。为了方便观察，这里使用了可视化表示方法。例如，左上角顶点的值“ $2 \uparrow$ ”实际上表示的是一个二元数对“ $(2, 90)$ ”，表示梯度的幅度为 2，角度为 90° 。

2↑	3↑	2↑	2↑	5↑
3↑	2↑	9↑	6↑	2↑
4↑	8↑	6↑	3↑	3↑
7↑	2↑	2↑	2→	4↗
6↑	2↑	2←	1→	2→

图 10-2 梯度示例

3. 非极大值抑制

在获得了梯度的幅度和方向后，遍历图像中的像素点，去除所有非边缘的点。在具体实现时，逐一遍历像素点，判断当前像素点是否是周围像素点中具有相同梯度方向的最大值，并根据判断结果决定是否抑制该点。通过以上描述可知，该步骤是边缘细化的过程。针对每一个像素点：

- 如果该点是正/负梯度方向上的局部最大值，则保留该点。
- 如果不是，则抑制该点（归零）。

在图 10-3 中，A、B、C 三点具有相同的方向（梯度方向垂直于边缘）。判断这三个点是否为各自的局部最大值：如果是，则保留该点；否则，抑制该点（归零）。

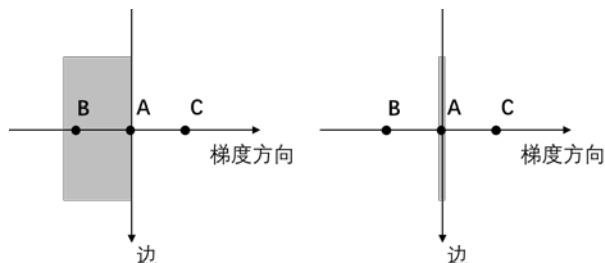


图 10-3 非极大值抑制示例之一

经过比较判断可知，A 点具有最大的局部值，所以保留 A 点（称为边缘），其余两点（B 和 C）被抑制（归零）。

在图 10-4 中，黑色背景的点都是向上方向梯度（水平边缘）的局部最大值。因此，这些点会被保留；其余点被抑制（处理为 0）。这意味着，这些黑色背景的点最终会被处理为边缘点，而其他点都被处理为非边缘点。

2↑	3↑	2↑	2↑	5↑
3↑	2↑	9↑	6↑	2↑
4↑	8↑	6↑	3↑	3↑
7↑	2↑	2↑	2→	4↗
6↑	2↑	2←	1→	2→

图 10-4 非极大值抑制示例之二



“正/负梯度方向上”是指相反方向的梯度方向。例如，在图 10-5 中，黑色背景的像素点都是垂直方向梯度（向上、向下）方向上（即水平边缘）的局部最大值。这些点最终会被处理为边缘点。

2 ↑	3 ↑	2 ↑	2 ↑	5 ↑
3 ↓	2 ↓	9 ↑	6 ↑	2 ↑
4 ↑	8 ↓	6 ↓	3 ↓	3 ↓
7 ↑	2 ↑	2 ↑	2 →	4 ↗
6 ↑	2 ↑	2 ←	1 →	2 →

图 10-5 正/负梯度方向示例

经过上述处理后，对于同一个方向的若干个边缘点，基本上仅保留了一个，因此实现了边缘细化的目的。

4. 应用双阈值确定边缘

完成上述步骤后，图像内的强边缘已经在当前获取的边缘图像内。但是，一些虚边缘可能也在边缘图像内。这些虚边缘可能是真实图像产生的，也可能是由于噪声所产生的。对于后者，必须将其剔除。

设置两个阈值，其中一个为高阈值 `maxVal`，另一个为低阈值 `minVal`。根据当前边缘像素的梯度值（指的是梯度幅度，下同）与这两个阈值之间的关系，判断边缘的属性。具体步骤为：

- （1）如果当前边缘像素的梯度值大于或等于 `maxVal`，则将当前边缘像素标记为强边缘。
- （2）如果当前边缘像素的梯度值介于 `maxVal` 与 `minVal` 之间，则将当前边缘像素标记为虚边缘（需要保留）。
- （3）如果当前边缘像素的梯度值小于或等于 `minVal`，则抑制当前边缘像素。

在上述过程中，我们得到了虚边缘，需要对其做进一步处理。一般通过判断虚边缘与强边缘是否连接，来确定虚边缘到底属于哪种情况。通常情况下，如果一个虚边缘：

- 与强边缘连接，则将该边缘处理为边缘。
- 与强边缘无连接，则该边缘为弱边缘，将其抑制。

在图 10-6 中，左图显示的是三个边缘信息，右图是对边缘信息进行分类的示意图，具体划分如下：

- A 点的梯度值大于 `maxVal`，因此 A 是强边缘。
- B 和 C 点的梯度值介于 `maxVal` 和 `minVal` 之间，因此 B、C 是虚边缘。
- D 点的梯度值小于 `minVal`，因此 D 被抑制（抛弃）。

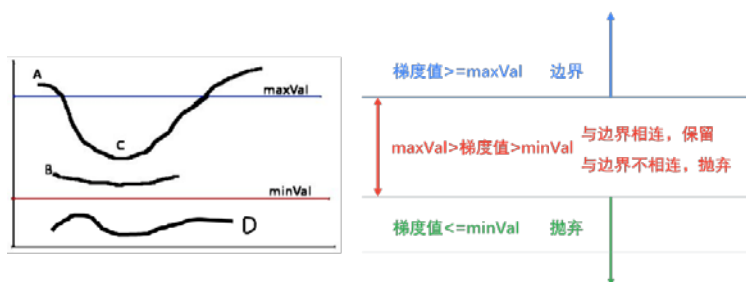


图 10-6 边缘分类的示意图

图 10-7 显示了对图 10-6 中的虚边缘 B 和 C 的处理结果。其中：

- B 点的梯度值介于 $\maxVal$ 和 $\minVal$ 之间，是虚边缘，但该点与强边缘不相连，故将其抛弃。
- C 点的梯度值介于 $\maxVal$ 和 $\minVal$ 之间，是虚边缘，但该点与强边缘 A 相连，故将其保留。

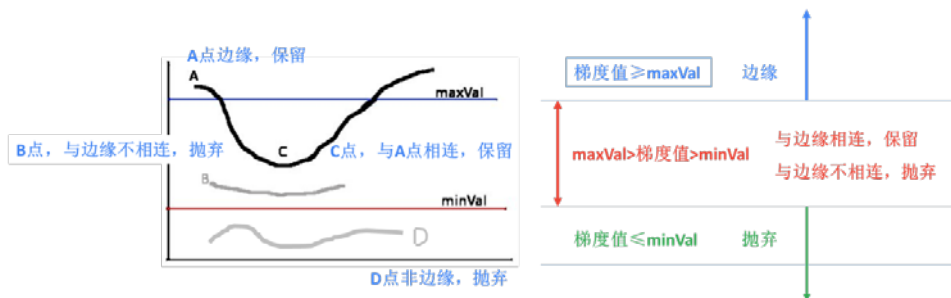


图 10-7 边缘分类的处理结果示意图

注意，高阈值 $\maxVal$ 和低阈值 $\minVal$ 不是固定的，需要针对不同的图像进行定义。

图 10-8 给出了一个 Canny 边缘检测的效果图。



图 10-8 Canny 边缘检测效果示例

10.2 Canny 函数及使用

OpenCV 提供了函数 `cv2.Canny()` 来实现 Canny 边缘检测，其语法形式如下：

```
edges = cv.Canny( image, threshold1, threshold2[, apertureSize[, L2gradient]])
```

式中：



- edges 为计算得到的边缘图像。
- image 为 8 位输入图像。
- threshold1 表示处理过程中的第一个阈值。
- threshold2 表示处理过程中的第二个阈值。
- apertureSize 表示 Sobel 算子的孔径大小。
- L2gradient 为计算图像梯度幅度（gradient magnitude）的标识。其默认值为 False。如果为 True，则使用更精确的 L2 范数进行计算（即两个方向的导数的平方和再开方），否则使用 L1 范数（直接将两个方向导数的绝对值相加）。

$$\text{norm} = \begin{cases} |dI/dx| + |dI/dy| & , \text{L2gradient} = \text{False} \\ \sqrt{(dI/dx)^2 + (dI/dy)^2} & , \text{L2gradient} = \text{True} \end{cases}$$

【例 10.1】使用函数 cv2.Canny() 获取图像的边缘，并尝试使用不同大小的 threshold1 和 threshold2，观察获取到的边缘有何不同。

根据题目要求，编写程序如下：

```
import cv2
o=cv2.imread("lena.bmp",cv2.IMREAD_GRAYSCALE)
r1=cv2.Canny(o,128,200)
r2=cv2.Canny(o,32,128)
cv2.imshow("original",o)
cv2.imshow("result1",r1)
cv2.imshow("result2",r2)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 10-9 所示。其中：

- (a)图是原始图像。
- (b)图是参数 threshold1 为 128、threshold2 为 200 时的边缘检测结果。
- (c)图是参数 threshold1 为 32、threshold2 为 128 时的边缘检测结果。



图 10-9 不同参数设置的边缘检测结果

从程序运行结果可知，当函数 cv2.Canny() 的参数 threshold1 和 threshold2 的值较小时，能够捕获更多的边缘信息。